

4. Resource and Process Management

Q.1 Explain desirable features of global scheduling algorithm.

⇒

- Following are the desirable features of good global scheduling algorithm.

1) No priori knowledge about the processes

⇒ Process scheduling algorithm should not consider a priori knowledge about the characteristics and resources need of the processes.

- It imposes extra burden on users as they should specify this information while submitting the job for execution.

2) Dynamic in nature

⇒ Process allocations should be able to mean that allocations should be made ~~easy~~ based on the current load of the system but not based on specific planned conditions.

- The scheduler should also be able to move processes from one node to another duplicate so that distribution is based on the current system load.

3) Quick decision making capability

⇒ Scheduling algorithms should be fast and should provide the best optimal decision in less amount of time.

- A good process-scheduling algorithm must make quick decisions about the assignment of processes to processors.

- This is an extremely important aspect of the algorithms and makes many potential solutions unsuitable.

4) Fault tolerance

⇒ A good global scheduling algorithm should not be stopped when system nodes are crashed or temporarily crashed.

- Algorithm configuration should also be even if the nodes are separated by multiple nodes.

5) Scalability

⇒ Process scheduling algorithm should work for small as well as large networks.

- If algorithm requires enquires for lightly loaded node and send processes there then it can work better for smaller networks.

- In large network as many replies needs to be processed it consumes bandwidth as well as processing power.

Q.2 Explain different load estimation and process transfer policies used by load balancing algorithms.

⇒

- Load balancing algorithm employ various load estimation policies and process transfer policies, to distribute the workload evenly across multiple nodes or resources in a distributed system.
- These policies plays a crucial role in determining how the algorithm estimates the load on each node.
- Also how it takes transfer processes or tasks from heavily loaded nodes to lightly loaded nodes.
- Load Estimation Policies

- 1) CPU Utilization
- 2) Memory Utilization
- 3) Queue length
- 4) Combination Policies

1) CPU Utilization

⇒ This policy estimates the load based on the CPU utilization of each node.

Nodes with higher CPU utilization are considered more heavily loaded.

For eg., Node A has 90% CPU usage vs Node B at 30%. The scheduler may move jobs from A to B.

2) Memory utilization

- ⇒ This policy considers the memory usage of each node to estimate the load.
- Nodes with higher memory consumption are deemed more heavily loaded.
 - For e.g., Node C using 95% memory is more loaded than Node D using 40%.

3) Queue length

- ⇒ In this policy, the load estimation is based on the length of the job or task queue at each node.
- Nodes with longer queues are considered more heavily loaded.
 - For e.g., Node E has 15 tasks waiting, Node F has only 2 tasks. Load balancer shifts some jobs to F.

4) Combination Policies

- ⇒ Some algorithms use a combination of multiple factors, such as CPU utilization, memory usage and queue length, to estimate the overall load on the node.
- Uses a weighted score:

$$\text{Load} = \alpha \cdot (\text{CPU}) + \beta \cdot (\text{memory}) + \gamma \cdot (\text{Queue length})$$

- For e.g., Node A has moderate CPU but very long queue and high memory use - classified as overloaded.

Process transfer policies

- 1) Transfer policy
- 2) Location policy
- 3) Selection policy.

1) Transfer Policy

⇒ This policy determines when and how processes or tasks should be transferred from heavily to lightly loaded nodes.

- a) Threshold-based: Processes are transferred when the load on a node exceeds a predefined threshold.
- b) Periodically: Processes are transferred at regular intervals, regardless of the load estimation.
- c) On-demand: Processes are transferred only when a lightly loaded node requests additional work.

2) Location Policy

⇒ This policy determines the destination nodes for the transferred processes or tasks.

Some common location policies include:

- a) Random: Destination nodes are selected randomly.
- b) Least loaded: Processes or tasks are transferred to the least loaded nodes.

c) Proximity-based : Processes or tasks are transferred to nodes that are geographically or logically closer to the source node, minimizing network overhead.

3) Selection Policy

⇒ This policy determines when specific processes or tasks should be transferred from a heavily loaded node.

Some common policies include:

a) Random : Processes or tasks are selected randomly for transfer.

b) Prioritized : Processes or tasks are selected based on priorities assigned to them.

c) Shortest job first : The shortest or smallest job is selected for transfer first.

d) Longest job first : The longest or largest job is selected for transfer first.

• The choice of load estimation and process transfer policies depends on various factors, such as the system's workload characteristics, resource constraints and performance constraints.

• Different combinations of these policies can be employed to achieve better load balancing and overall system performance.

Q.3 Discuss the different issues and steps involved in a good load balancing algorithm.

⇒ Following are issues related to designing load-balancing algorithms:

1) Dynamic Workload Variations

⇒ Workloads in distributed system can vary dynamically based on factors like user traffic patterns, time of day or seasonal variations.

- Load balancer must continuously monitor and adapt to these changes to distribute incoming requests efficiently across nodes without causing overload.

2) Scalability

⇒ As distributed systems scales up or down dynamically, load balancers must scale along-side to handle increased traffic and resource demands.

- Implementing auto-scaling mechanisms and integrating with cloud services to add or remove nodes based on workload changes, ensuring seamless performance.

3) Overhead

⇒ Load balancing algorithms involve computational overhead and complexity in decision-making.

- Efficient balancing must minimize these costs.

4) Network latency

⇒ Distributed systems operate over networks where nodes may be geographically dispersed, leading to varying network latencies and bandwidth constraints.

- Load balancer need to consider network conditions to minimize response time and optimize data transfers between nodes while ensuring balanced resource utilization.

5) Security

⇒ Distributed system must adhere to security policies and regulatory compliance requirements regarding data protection, access control and secure communication.

- Load balancers must need robust security features such as SSL termination, encryption and protection against DDoS attacks.

• Steps:

1) Resource Monitoring

⇒ Continuously collect data on the current load of each server or node.

- This can be centralized or distributed.

2) Load Evaluation

⇒ Evaluate if the system is imbalanced.

- Use a load metric such as CPU load, no. of tasks, response time.

- If imbalance is detected, decide which task should be moved, from where and to where.

3) Load balancing policy

⇒ Choose a policy that governs how the decision is made.

- Common types:

a) Round Robin: Assign tasks in a circular order.

b) Least Connections: Sends tasks to the server with the fewest active connections.

c) Least loaded: uses a real-time metrics to choose the least loaded server.

d) Random: Assigns to a random node.

e) Weighted: Assigns tasks based on server weights.

4) Task Assignment

⇒ Based on the decision, assign incoming tasks or migrate existing ones.

- This may involve:

i) Redirecting new requests.

ii) Migrating data or ongoing tasks.

5) Feedback

⇒ Update the state information after load is assigned.

- Feed this back into the monitoring system to refine future decisions.

Q.4 Compare static and dynamic load balancing algorithms.

Static load balancing	Dynamic load balancing
1) Designed for the system with low fluctuation in incoming load.	1) Designed for the system with high fluctuation in incoming load.
2) Traffic is divided equally among the servers.	2) Traffic is dynamically divided among the servers.
3) It requires deeper information about available system resource.	3) It does not necessarily need deeper information.
4) It does not require real-time communication with the servers.	4) It requires real-time communication actively with the servers.
5) The allocated load cannot be retransferred to other servers during runtime.	5) The allocated load can be retransferred among servers to reduce the under utilization of resources.
6) For e.g., Round Robin algorithm.	6) For e.g., least connection algorithm.

Q.5 Explain with an example load sharing approach.

⇒

- In load sharing algorithm, all nodes share the overall workload, and the failure of some nodes increases the pressure of the rest of the nodes.

- The load sharing approach ensures that no node is kept idle so that each node can share the load.

- There are several issues in designing load-balancing approach. To overcome these issues we use the load-sharing algorithm.

- Load sharing algorithm includes policies like location policy, process transfer policy, state information exchange policy.

1) Load Estimation Policies

⇒ Load sharing algorithms only ensure that no node is idle.

- Hence, simplest policy to estimate load is 'total number of processes or nodes'.

- As in modern distributed system, several processes permanently reside on idle node, CPU utilization is considered as measure to estimate load on node.

2) Process transfer policies

⇒ Load-sharing algorithms mainly concern with two states of nodes: busy or idle.

- It uses single threshold policy with threshold value of 1.

- Node accepts process if it has no process. Node transfer the process as soon as it has more than two processes.
- If node becomes idle, it can be unable to accept new process immediately.
- This policy wastes the processing power of idle node.
- The solution to above problem is to transfer the process to the node which is expected to become idle.

3) Location Policies

→ In load-sharing algorithm, sender or receiver node is involved in load sharing process.

a) Sender-initiated policy: The overloaded node initiates the process transfer by searching for lightly loaded node. It is suitable for lightly loaded systems, as finding a free node is easier.

b) Receiver-initiated policy: The underloaded node initiates the request by searching for overloaded node. It is effective in heavily loaded system, where senders are abundant.

4) State-information exchange policies

⇒ In load-sharing approach, state information exchange is carried out only when node changes the state.

— Because, node needs to know state of other node when it becomes overloaded or underloaded.

— In this case following policies are used:

a) Broadcast when state changes: In this policy node broadcasts state information request when its state changes. In sender-initiated policy, the state information request is only broadcasted by the node when node is overloaded and in receiver-initiated policy when a node is underloaded.

b) Poll when state changes: In a large network the polling operation is performed. It arbitrarily asks different nodes for state information till it gets an appropriate one or it reaches the test limit.

- E.g., Imagine a distributed system with 4 nodes: P_1, P_2, P_3, P_4 .
- Each can process 3 tasks simultaneously.

Node	Task Assigned
P_1	6 (overloaded)
P_2	2
P_3	1
P_4	2

- Action taken:
- P_1 detects it is overloaded.
- It requests help from P_3 and P_4 .
- P_3 agrees to take 2 tasks, P_4 takes 1 task.
- After load sharing:

Node	Task Assigned
P_1	3
P_2	2
P_3	3
P_4	3

Q.6 What is the need for Code Migration?
Explain the role of process to resource and resource to machine binding in Code Migration

⇒

- Code migration involves transferring software code from one environment to another.
- Need for Code Migration:

1) Dynamic environment

⇒ Code migration allows systems to adapt to frequent changes by transferring code to different locations as needed.

2) Resource Optimization

⇒ It optimizes resource utilization by moving code to nodes with excess capacity, balancing workload.

3) System Maintenance

⇒ Facilitates seamless deployment of updates, ensuring applications remain functional.

4) Scaling

⇒ Enables dynamic resource allocation to handle increased traffic and workload demands.

5) Fault tolerance

⇒ Aids in quick recovery from failures by migrating codes to ensure continuity of services.

Role of Binding in Code Migration:

~~three~~

1) Process-to-resource Binding

⇒ This determines how tightly a process depends on a particular resource.

- Three types of process-to-resource binding exist.

- Binding by identifier requires precisely the referenced resource such as URL to refer to a specific website.

- References to local communication end points are also in this category.

- In Binding by value, only the value of a resource is needed.

- In this binding, execution of the process does not affect if same value is available on target node.

- A typical example of binding by value is libraries that are locally available.

- In binding by type, a process indicates it needs only a resource of a specific type.

- Example is references to local devices, such as monitors, printers and so on.

2) Resource-to-machine Binding

⇒ This relates to how tightly the resource is linked to a specific machine.

- Unattached resources can be easily moved from one machine to other.

- The example is 'data file'.
- On the other hand, it is possible to move fastened resources but incurring high cost.
- For e.g., local databases and complete web sites.
- Fixed resources like local devices and communication endpoints cannot be moved.
- Considering above types of both, following combination is required to consider while migrating the code.

Process to resource binding	Unattached resource	Fastened resource	Fixed resource
By Identifier	Move	Global ref	Global ref
By value	Copy	Global ref	Global ref
By Type	Rebind	Rebind	Rebind

1) Move: Move the resource

2) Global ref: Establish a global system wide references.

3) Copy: Copy the value of resource.

4) Rebind: Rebind process to locally available resource.

Q.2 Discuss code migration in distributed system.

- ⇒
- Code migration involves transferring software code from one environment to another.
 - It enables better load balancing, data locality, performance and flexibility.

• Code Migration Issues:

1) Binding and Resource Access:

- ⇒ Code may lose access to required files, devices or data after migration.
- Pre-migration bindings may not be valid at the destination.
 - Needs strategies like late binding or rebinding.

2) Execution state transfer

- ⇒ For strong migration, stack, heap and programs counter must be preserved.
- State serialization is complex and often platform-dependent.
 - Not all languages support capturing full execution context.

3) Security risks

- ⇒ Host might execute malicious incoming node.
- Code may be tampered with or exposed during transfer.
 - Requires authentication, authorization.

4) Platform Heterogeneity

⇒ Sources and destination machines may differ in architectures or OS.

- Binary code might not run across platform.

- Requires use of platform-independent code.

5) Network Reliability

⇒ Migration depends on stable network connections.

- Packet loss or delays may corrupt code/state during transfer.

- Must implement error checking and retries.

• Techniques of Code Migration

1) Process Migration

⇒ Process migration involves moving an entire running process, including its code, data and execution state, from one node to another within the distributed system.

- The process is typically suspended, its state is transferred to the destination node, and then it is resumed on new node.

- Process migration useful for load balancing, fault tolerance and dynamic resource allocation.

2) Thread Migration

- ⇒ Thread migration focuses on moving individual threads of execution between different nodes within the distributed system.
- Threads are typically suspended, their execution state is transferred to the destination node, and they are resumed on the new node.
- Thread migration is beneficial for optimizing resource usage and improving parallelism.

3) Object Migration

- ⇒ Object migration involves transferring objects or components of application between different nodes within the distributed system.
- Objects are serialized, transmitted to the destination node, and then deserialized to recreate the objects in the new environment.
- Objects migration is commonly used for data replication, caching and distributed computing.

4) Container Migration

- ⇒ Container migration focuses on moving containerized applications or services between different hosts or clusters within the distributed system.
- Containers are stopped, their state is captured, and then they are restarted on the destination node or cluster.

- Containers migration enables flexible deployment, scaling and resource management.

5) Virtual Machine Migration

- ⇒ Virtual machine migration involves transferring entire virtual machines, including their operating system and applications, between different physical hosts within the distributed system.
- Virtual machine state is captured using live migration techniques and transferred to the destination node.
- Virtual machine migration facilitates workload consolidation, disaster recovery and resource optimization.

Q.8 Justify how load balancing is useful in distributed system?

⇒

- A load balancer is a device that acts as reverse proxy and distributes network or application traffic across a number of servers.

- Key Benefits:

1) Improved Resource Utilization

⇒ Ensures all nodes are used efficiently.

- Prevents overloading of any single node while others remain underutilized.

- For e.g., If 3 out of 10 servers are overburdened while 7 are idle, load balancing redistributes tasks.

2) Enhanced system performance

⇒ Reduces response time by distributing requests evenly.

- Minimizes 'queue' lengths and processing delays.

- For e.g., Web applications use load balancers to distribute user requests across multiple backend servers for faster response.

3) Scalability

⇒ One of the key advantage of distributed system is horizontal scalability, you can add more machines as demand grows.

- A load balancer abstracts the addition of new node.

- When a new server joins the system, the load balancer starts sending tasks to it automatically.

4) Fault tolerance and Reliability

- ⇒ IF one node fails, tasks can be reassigned to others.
- Maintains service availability and prevents single point of failure.
- Load balancer performs health checks and detect when a node becomes unresponsive or fails.

5) Cost efficiency

- ⇒ Running a large number of nodes without managing the load efficiently leads to wasted resources.
- In cloud environments (where you pay per use), efficient load distribution helps you minimize costs by utilizing existing resources fully before new ones.

Q.11 Explain in detail about process migration.

=>

- Process migration is transfer of process from its current node which is source node to destination node.

- There are two types of process migration:-

1) Non-preemptive process migration

⇒ In this approach, process is migrated from its current to destination node before it starts executing on current node.

2) Preemptive process migration

⇒ In this, process is migrated from its current to destination node during its course of execution on current node.

- Key concepts

1) Process state: The complete status of a process, including its memory contents, register values, program counter and open file descriptors, that must be captured and transferred during migration.

2) Checkpointing: The act of saving the current state of a process to enable resumption from that point after migration.

3) Transparency: Making the migration process seamless so that the process and its users do not notice the transition.

4) Fault tolerance - Mechanism to handle failures during migration, ensuring that the process can be restarted or resumed without loss of critical data.

Steps:

1) Selection of Process for Migration

- ⇒ Identify the process that needs to be migrated based on criteria such as load balancing, resource optimization or fault tolerance.
- Evaluate the process's resource usage, current load on the source node, and potential benefits of migration.

2) Choosing the destination node

- ⇒ Select the appropriate destination node where the process will be relocated.
- Consider Factors like available resources, compatibility, network resources latency and current load on potential destination nodes.

3) Migrating the Process to Destination Node

- ⇒ Transfer the process from the source node to the destination node.
- This involves several subcategories of migration, each addressing different aspects of the process's state and execution.